快速入门STM32HAL库&&蓝桥杯嵌入式

鸣谢B站UP：我的代码没问题

/*

- @Author: 洛 (Loren_zzi)
- @Date: 2026-04-02
- @Description:快速入门STM32HAL库&&蓝桥杯嵌入式
  */

代码区域：

因为 CubeMX 是一个图形化配置工具，当你修改了引脚配置并点击“Generate Code”时，它会重新生成 `main.c`。**在 CubeMX 生成的文件中，自定义代码应放在原有的 `USER CODE BEGIN/END` 区域，并勾选 `Keep User Code when re-generating`，否则重新生成时可能被覆盖。**自己创建且不由 CubeMX 生成的 .c/.h 文件不受这些标记限制。

```
/* Private typedef -----------------------------------------------------------
*/
/* USER CODE BEGIN PTD */
PTD(Private Typedef) -- 私有类型定义
/* USER CODE END PTD */

/* Private define ------------------------------------------------------------
*/
/* USER CODE BEGIN PD */
`PD` (Private Define) -- 私有宏定义
/* USER CODE END PD */

/* Private macro -------------------------------------------------------------
*/
/* USER CODE BEGIN PM */
`PM` (Private Macro) -- 私有宏函数
/* USER CODE END PM */

/* Private variables ---------------------------------------------------------
*/

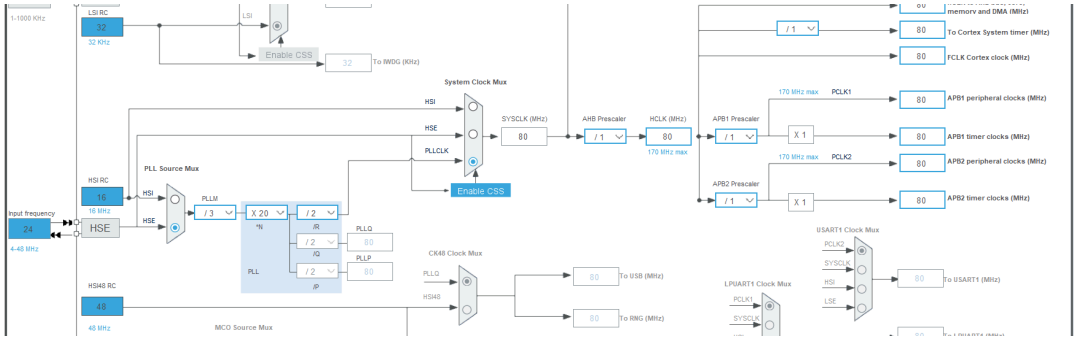
/* USER CODE BEGIN PV */
`PV` (Private Variables) -- 私有变量
```

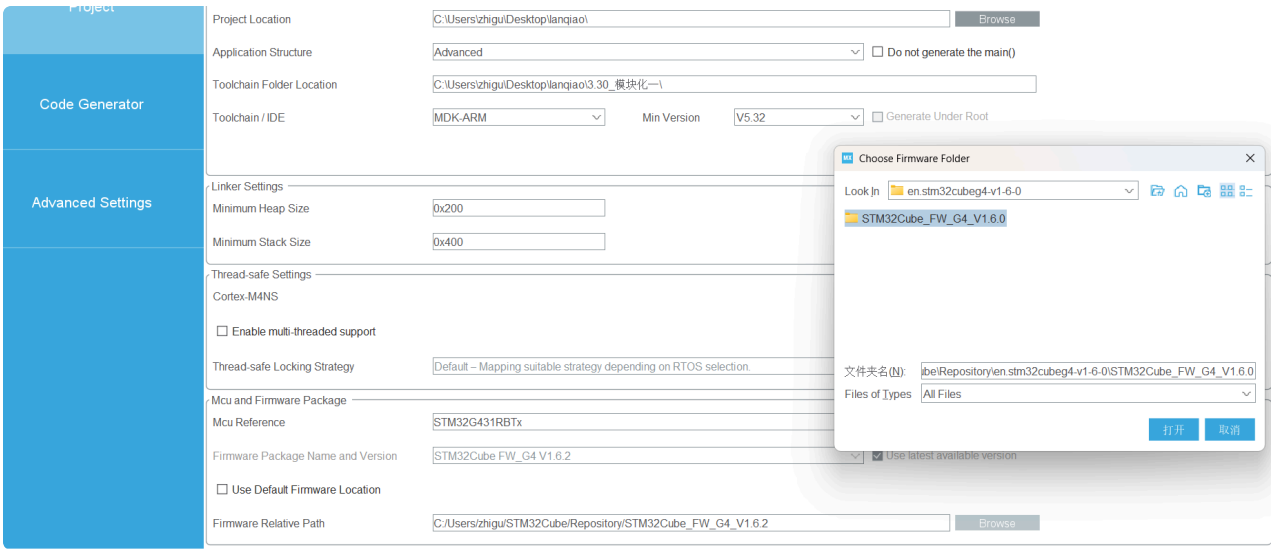
```
/* USER CODE END PV */

/* Private function prototypes -----------------------------------------------
*/
void SystemClock_Config(void);
/* USER CODE BEGIN PFP */
(Private Function Prototypes)
/* USER CODE END PFP */
```

点点点：操作

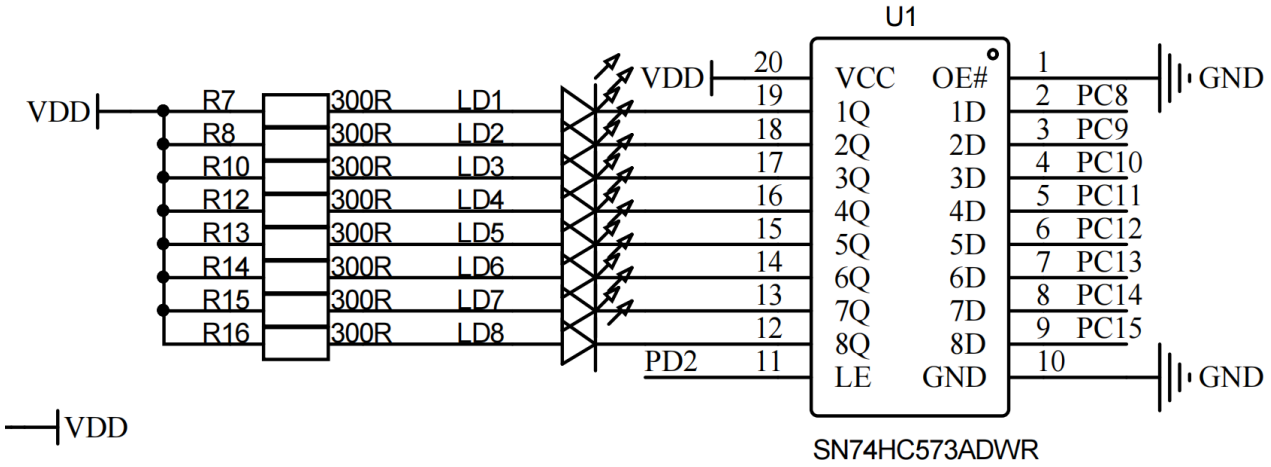
1. （RCC）：外部时钟使能外部高速时钟
2. （SYS）：Debug 选择 Serial Wire（SWD 调试接口，不是 UART 串口）。
3. 系统时钟配置：





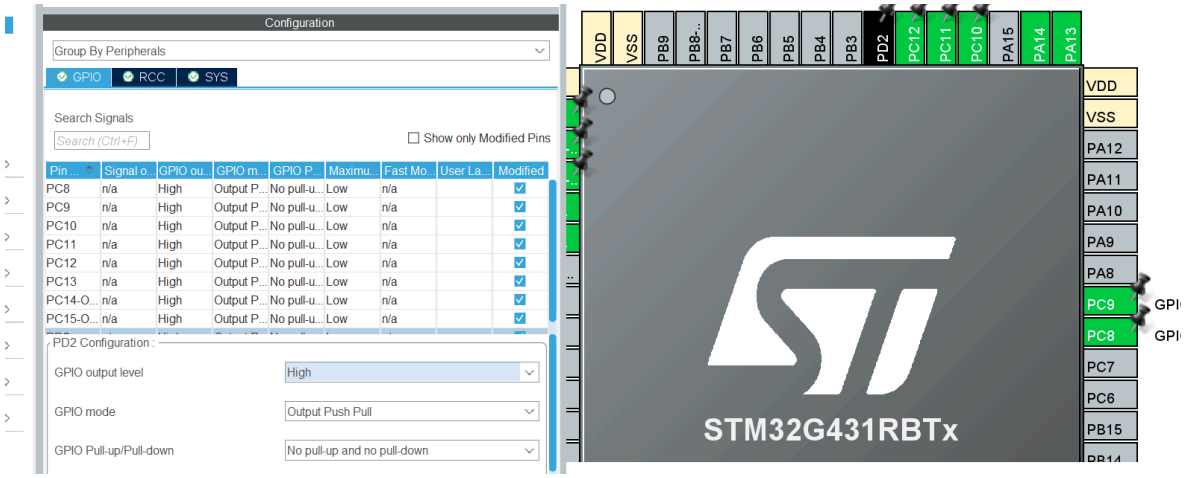
4. 并且注意：在Code Generator选择生成.c.h文件
5. 魔术棒 debug选择Reset and Run
6. 增加code分组，也要在魔术棒里添加路径

GPIO、LED




```
void show_led(uint8_t led,uint8_t state)
{
    HAL_GPIO_WritePin(GPIOD,GPIO_PIN_2,GPIO_PIN_SET);
    if(state==1)
    {
        HAL_GPIO_WritePin(GPIOC,0x0080<<led,GPIO_PIN_RESET);
    }
    else
    {
        HAL_GPIO_WritePin(GPIOC,0x0080<<led,GPIO_PIN_SET);
    }
    HAL_GPIO_WritePin(GPIOD,GPIO_PIN_2,GPIO_PIN_RESET);
}
```

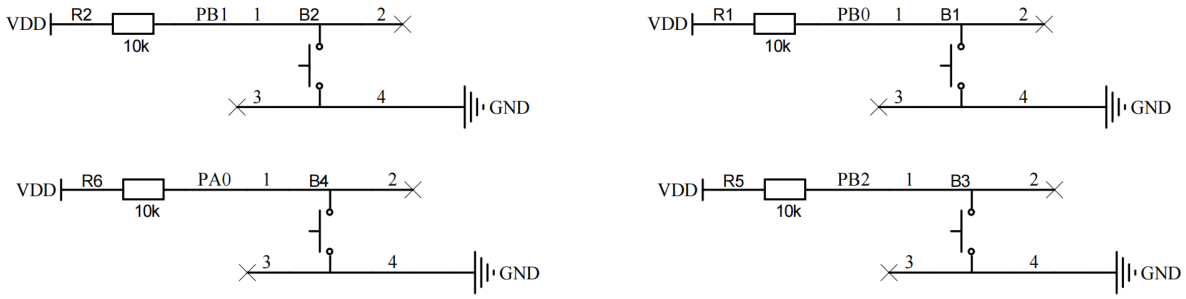
初始化：PC8-PC15 高电平、推挽输出，LED 为低电平点亮。74HC573 是锁存器，不是译码器；PD2 为高电平时输出跟随输入，为低电平时保持锁存数据。为避免与 LCD 共用引脚时相互影响，GPIO 初始化时先将 PD2 设为低电平。



PD2 的角色： 它连接到 74HC573 的 **LE (LATCH ENABLE)** 引脚，即“锁存使能端”。
如果不及时拉低 PD2（锁存数据），当调用 LCD 刷新函数时，LCD 的数据信号会传到 GPIOC，

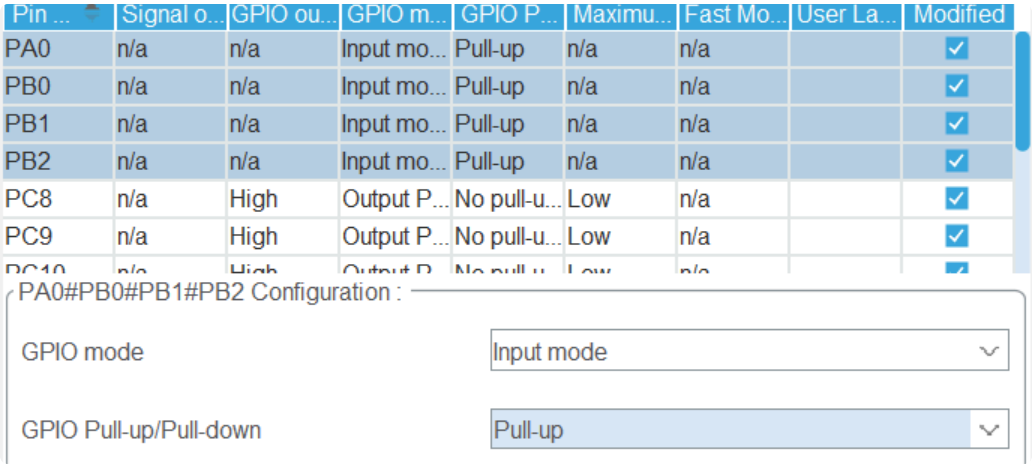
由于锁存器还是打开的，你的 **LED 就会跟着 LCD 的数据乱闪。*！！所以每次操作完LED就要立即将PD2拉低！***

按键：



初始化：PB0、1、2、PA0为上拉输入模式

当HAL_GPIO_ReadPin读到为0时，说明按键按下



下面介绍一种非阻塞式的写法：

状态机逻辑本质是检测按下是否达到长按键阈值、否则是短按键

中断修改、主循环读取的状态变量应加 `volatile`，避免编译器优化掉必要的访问；但它不保证“读取后清零”等多步操作的原子性。

``

```
//////////////////////////////////////////在key.c中
//============================key
// 定义全局变量时加上 volatile
//在中断和主循环中共同访问的变量，必须加 `volatile` 防止被编译器优化掉
volatile uint8_t key_short;
volatile uint8_t key_long;

static uint8_t key_time=0;
static uint8_t long_state=1;
```

```c
uint8_t key_state(void)
{
    if(HAL_GPIO_ReadPin(GPIOB,GPIO_PIN_0)==0) return 1;
    if(HAL_GPIO_ReadPin(GPIOB,GPIO_PIN_1)==0) return 2;
    if(HAL_GPIO_ReadPin(GPIOB,GPIO_PIN_2)==0) return 3;
    if(HAL_GPIO_ReadPin(GPIOA,GPIO_PIN_0)==0) return 4;

    return 0;
}

void key_loop(void)
{
    static uint8_t last_state=0,now_state=0;

    last_state=now_state;
    now_state=key_state();

    uint8_t i;
    for(i=1;i<=4;i++)
    {
        if(last_state==0&&now_state==i)
        {
            key_time=0;
            long_state=1;    //重置长按标志位
        }

        if(last_state==i&&now_state==i)
        {
            if(++key_time==40&&long_state==1)
            {
                key_long=i;
                long_state=0;
            }
        }

        if(last_state==i&&now_state==0)
        {
            if(long_state==1)
            {
                key_short=i;
            }
            else
            {
                long_state=1;
            }
```

```
        }
    }


}
//////////////////////////////在定时器中每20ms刷新一次函数key_loop
uint8_t led7_state;
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    static uint16_t count_1s,count_20ms;
    if(htim->Instance==TIM3)  //10ms
    {
        if(++count_1s==100)
        {
            count_1s=0;
            led7_state++;
            led7_state=led7_state%2;
        }
        if(++count_20ms==2)
        {
            count_20ms=0;
            key_loop();
        }

    }

}
//////////////////////////////////////在主函数while里调用：
      if(key_short)
      {
        switch(key_short)
        {
            case 1:
            {
                count1++;
            }break;

            case 3:
            {
                led2_state=(++led2_state)%2;
                show_led(2,led2_state);
            }break;

            case 4:
            {
                count2++;
```

```
            }break;

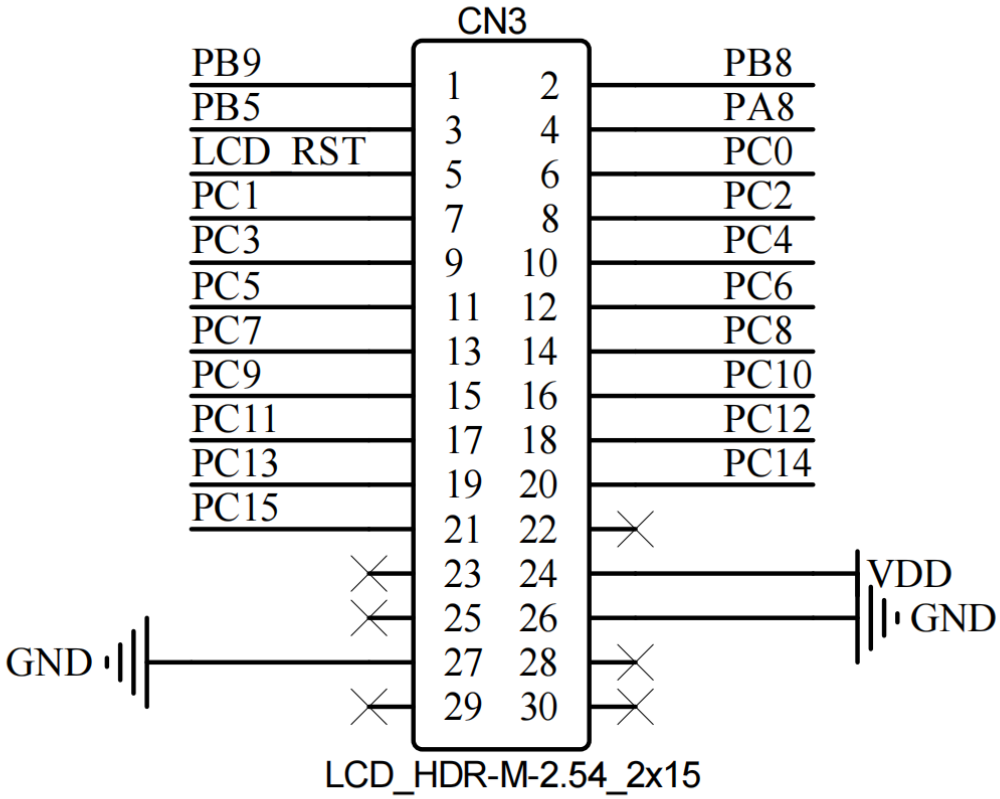
        }key_short=0;
    }

        if(key_long)
    {
        switch(key_long)
        {
            case 1:
            {
                count1--;
            }break;

            case 4:
            {
                count2--;
            }break;

        }key_long=0;
    }
```

LCD：



初始化：

将Src中lcd.c、Inc中fonts.h好有lcd.h文件复制到我们工程code里面

在官方给的LCD.c文件里已经帮你初始化过，无需再手动配置

```
  LCD_Init();
  LCD_Clear(Black);
  LCD_SetBackColor(Black);
  LCD_SetTextColor(White);
```

每一行最多显示 20 个字符；若字符串实际长度为 20，数组至少需要 21 字节，额外一字节用于结束符 `\0`。下面的 `text[20]` 只适用于实际字符串长度不超过 19 的情况。

```
char text[20];
void show_lcd(void)
{
    sprintf(text,"       text:");
    LCD_DisplayStringLine(Line0,(u8 *)text);
    sprintf(text,"    count:  %-3d",count);
    LCD_DisplayStringLine(Line1,(u8 *)text);
}
```

关于**残余字符**：%3d 或 %-3d 只是数字的最小字段宽度，超过三位仍会完整输出，也不会自动擦掉整行。显示内容变短时，需要补足空格或清行；整行高亮也需要把整行重新绘制。
为什么一个text变量就能显示、这是人眼视觉残留效应吗?
答：不是，是利用了 LCD 硬件自带的“显存”功能

某一行高亮：

`LCD_SetBackColor` 确实是设置一个全局变量（通常叫 `BackColor`），但它**不会立刻把已经画在屏幕上的东西变色**。它影响的是**在这行代码之后**执行的“打印”动作。

```
void show_lcd(void)
{
    // 第一步：Line0
    LCD_SetBackColor(highlight_line == 0 ? Yellow : Black);
    sprintf(text, "   hello,world!   ");
    LCD_DisplayStringLine(Line0, (u8*)text);

    // 第二步：Line1
    LCD_SetBackColor(highlight_line == 1 ? Yellow : Black);
    sprintf(text, "    count1:  %-3d", count1);
    LCD_DisplayStringLine(Line1, (u8*)text);

    // 第三步：Line2
    LCD_SetBackColor(highlight_line == 2 ? Yellow : Black);
    sprintf(text, "    count2:  %-3d", count2);
    LCD_DisplayStringLine(Line2, (u8*)text);

    // 最后记得切回黑色，防止影响其他地方
    LCD_SetBackColor(Black);
}
```

关于LED和LCD共用引脚冲突问题：

1. GPIO 初始化时建议将 PD2 设为低电平；若按下面的代码在初始化后手动拉低，必须在 LCD 初始化和刷新之前完成。

```
MX_GPIO_Init();
/* USER CODE BEGIN 2 */
HAL_GPIO_WritePin(GPIOD,GPIO_PIN_2,GPIO_PIN_RESET);
```

2. 在每一个关于LCD的函数内加上代码：

```
u16 temp = GPIOC->ODR; // 进门前：把 GPIOC 现在的状态（包含 LED 的亮灭情况）拍个照存下来
// ... 中间任由 LCD 驱动怎么折腾 GPIOC 都没关系 ...
GPIOC->ODR = temp;      // 出门前：按照刚才拍的照片，把 GPIOC 恢复原样

例如初始化：
void LCD_Init(void)
{
    u16 temp=GPIOC->ODR;
    LCD_CtrlLinesConfig();
    dummy = LCD_ReadReg(0);

    if(dummy == 0x8230)
    {
        REG_8230_Init();
    }
    else
    {
        REG_932X_Init();
    }
    dummy = LCD_ReadReg(0);
    GPIOC->ODR=temp;
}
```

3. 再加上你在调用 LCD 前，通过 `HAL_GPIO_WritePin(GPIOD,GPIO_PIN_2,GPIO_PIN_RESET);` 把锁存器关死了。这样 LCD 刷屏幕的时候，数据进不到 LED 里；等屏幕刷完，你又把 `GPIOC` 恢复到了刷屏幕前的状态。在 LED 和 LCD 不并发操作相关引脚的前提下，这样可以实现**分时复用**。

定时器的一些参数及其关系（定时器时钟、PSC、ARR、CCR）

要把定时器（Timer）玩透，理解这四个参数之间的“级联”关系是基本功。我们可以把定时器想象成一个**“往木桶里自动注水”**的过程。

1. 核心成员介绍

在 STM32 中，定时器的核心是由一个 **16位/32位计数器（CNT）** 驱动的。

参数名称	全称	形象比喻	作用
f_{TIM}	**定时器输入时钟 (Timer Clock)**	水流的速度	分频前的定时器时钟，不一定等于系统主频。
PSC	**预分频器 (Prescaler)**	水龙头开关（减速器）	决定计数器“跳动”一次需要多少时间。
ARR	**自动重装载寄存器 (Auto-Reload)**	木桶的高度	决定计数器数到多少就“溢出”重启，即**周期**。
CCR	**捕获/比较寄存器 (Capture/Compare)**	水桶里的浮标（阈值）	决定 PWM 的**占空比**或记录**捕获时刻**。

2. 它们之间的级联逻辑

第一步：分频 ($f_{TIM} \rightarrow PSC$)

定时器输入时钟较快时（比如 80MHz），可以用 PSC 降低计数频率。下面的 `f_TIM` 应从 CubeMX 时钟树中该定时器对应的 APB Timer Clock 读取，不能一律用系统主频代替；STM32G431 的 APB 分频为 1 时，定时器时钟等于对应 PCLK，APB 分频大于 1 时通常为对应 PCLK 的 2 倍。

- **频率计算**：计数器频率 $f_{CNT} = \frac{f_{TIM}}{PSC+1}$。
- **物理意义**：这就是你之前问的 $80-1$ 的由来。设为 79，则 $f_{CNT} = 1\text{MHz}$，意味着计数器每 $1\mu s$ 加 1。

第二步：决定周期 ($PSC \rightarrow ARR$)

计数器从 0 数到 ARR（含 ARR），下一计数节拍才回到 0，共 ARR+1 个节拍。以下公式适用于向上计数、边沿对齐，且没有重复计数器等额外分频的情况。

- **溢出频率**：$f_{overflow} = \frac{f_{CNT}}{ARR+1} = \frac{f_{TIM}}{(PSC+1)(ARR+1)}$。
- **应用**：这就是你输出 PWM 波形的频率，或者是定时器中断（如你的 `htim3`）触发的频率。

第三步：决定占空比 ($ARR \rightarrow CCR$)

*这是在 **PWM 模式**下最常用的逻辑。计数器 CNT 在从 0 到 ARR 增长的过程中，会不断和 CCR 比较。*

- **逻辑**：当 $CNT < CCR$ 时，引脚输出高电平；当 $CNT \geq CCR$ 时，引脚输出低电平（假设为 PWM mode 1、输出极性 High、向上计数且边沿对齐）。

- **占空比 (Duty Cycle)**：$Duty = \frac{CCR}{ARR+1}$。

$$f_{PWM} = \frac{f_{TIM}}{(PSC+1) \times (ARR+1)}$$

$$Duty = \frac{CCR}{ARR+1} \times 100\%$$

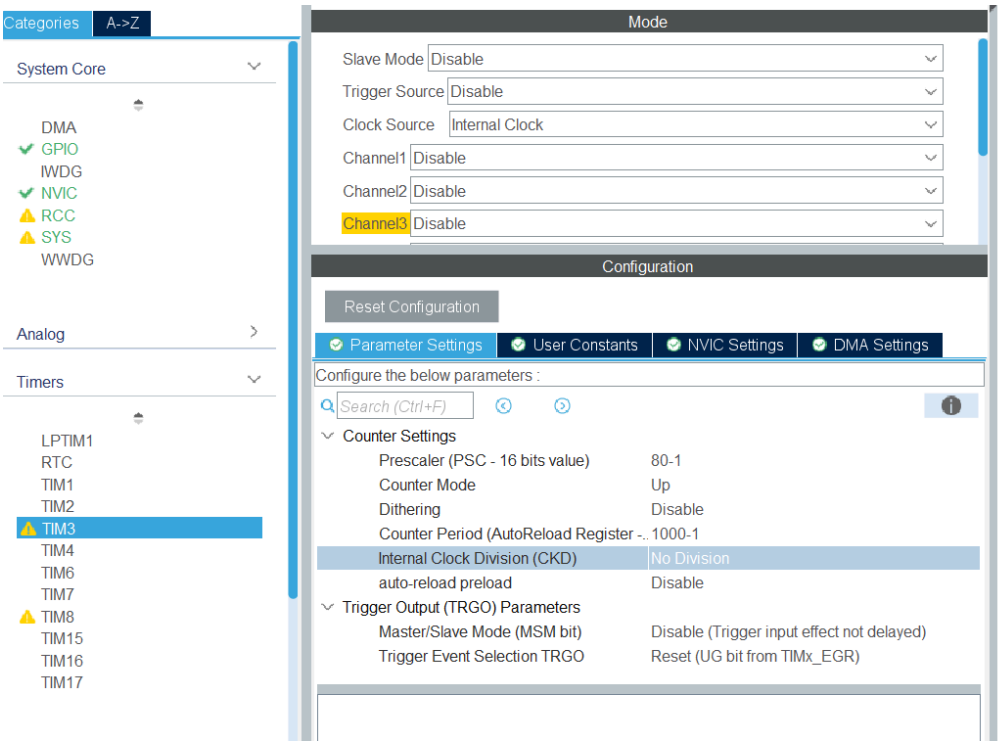
定时器：

以下数值例子假设该定时器的输入时钟为 80MHz，时钟不同需要重新计算。

$$f = 80000000/(psc+1)/(arr+1)$$

所以周期就是：
$T = \frac{(psc+1)\times(arr+1)}{80\ 000\ 000}$

一般配置10ms就够用了：
10ms psc=80-1 arr=10000-1
1ms: psc=80-1 arr=1000-1
配图为 1ms 配置，记住打开内部时钟和中断。但下方回调按 10ms 写：如果照图设成 1ms，`count_1s` 的阈值应为 1000、`count_20ms` 应为 20；使用原代码的 100 和 2 时，定时器必须配置为 10ms。



初始化使能

```
HAL_TIM_Base_Start_IT(&htim3);
```

使用回调函数：

```
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    static uint16_t count_1s,count_20ms;
    if(htim->Instance==TIM3)  //10ms
    {
        if(++count_1s==100)
        {
            count_1s=0;
            led7_state++;
            led7_state=led7_state%2;
        }
        if(++count_20ms==2)
        {
            count_20ms=0;
            key_loop();
        }

    }

}
```

PWM：

频率：

$$f = \frac{f_{TIM}}{(ARR+1)(PSC+1)}$$

CCR - 输出比较寄存器

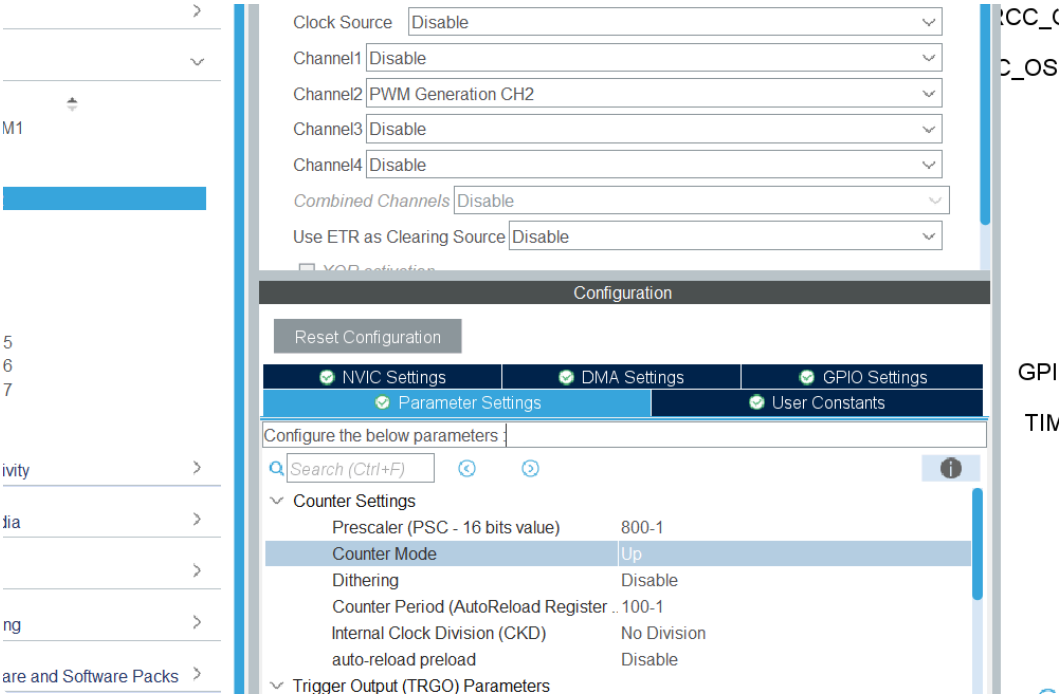
周期：

$$T = \frac{1}{f} = \frac{(ARR+1)(PSC+1)}{f_{TIM}}$$

占空比：

$$\frac{CCR}{ARR+1}$$

如果：让PA1输出频率为1000Hz，占空比为50%的方波
假设 TIM2 输入时钟为 80MHz，将 PA1 配置为 TIM2_CH2，PSC=800-1、ARR=100-1，输出频率就是 1000Hz；占空比计算按 PWM mode 1、极性 High、向上计数且边沿对齐。



main初始化中：设置占空比为百分之五十

```
HAL_TIM_PWM_Start(&htim2,TIM_CHANNEL_2);
TIM2->CCR2=50;
```

IC（输入捕获）：

```
$f_{CNT} = \frac{f_{TIM}}{PSC + 1}$
```

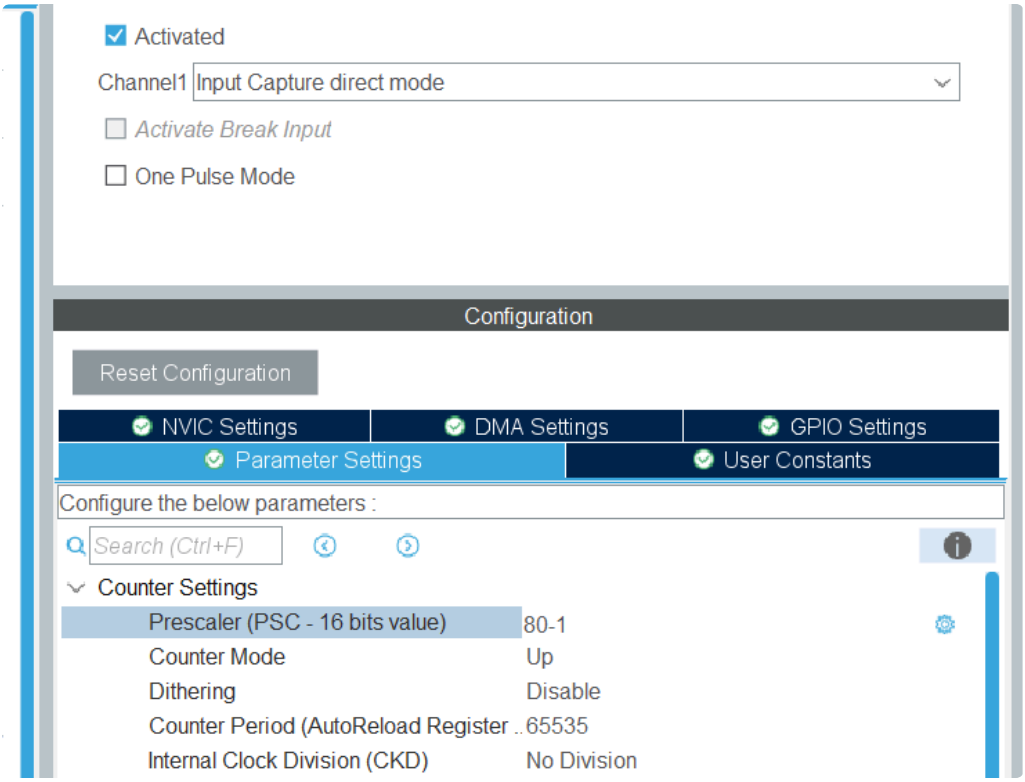
$$T = t_0 \cdot \text{capture_value}$$

$$f = \frac{1}{T} = \frac{f_{TIM}}{(PSC + 1) \cdot \text{capture_value}}$$

输入捕获引脚PWM频率：

利用PA7输入捕获上面的PA1输出的PWM：

1. cubemx配置PA7为TIM17定时器，记得要中断使能
   PSC 决定 CNT 的计数频率，ARR 决定计数回绕范围，也必须配置；若两次边沿之间发生未处理的溢出，测得的周期就会错误。



2，IC中断使能启动：

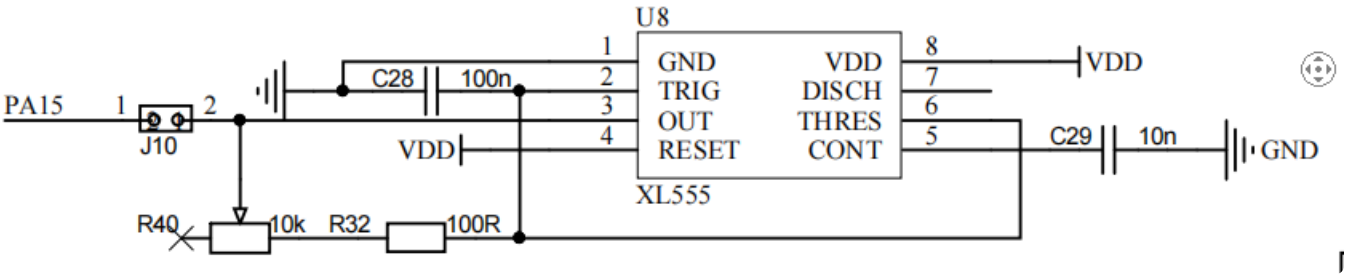
```
HAL_TIM_IC_Start_IT(&htim17,TIM_CHANNEL_1);
```

3，中断读取 CCR 赋值给 capture，并计算。下面清零 CNT 的写法是入门近似，会包含中断延迟误差；第一次捕获不能直接当成完整周期，capture 为 0 时也不能直接做除法。公式中的 capture_value 应代表一个周期的有效计数节拍数。

当外部信号的上升沿触发时，硬件会自动把那一瞬间计数器 `CNT` 的数值“锁存”到 `CCR1` 中

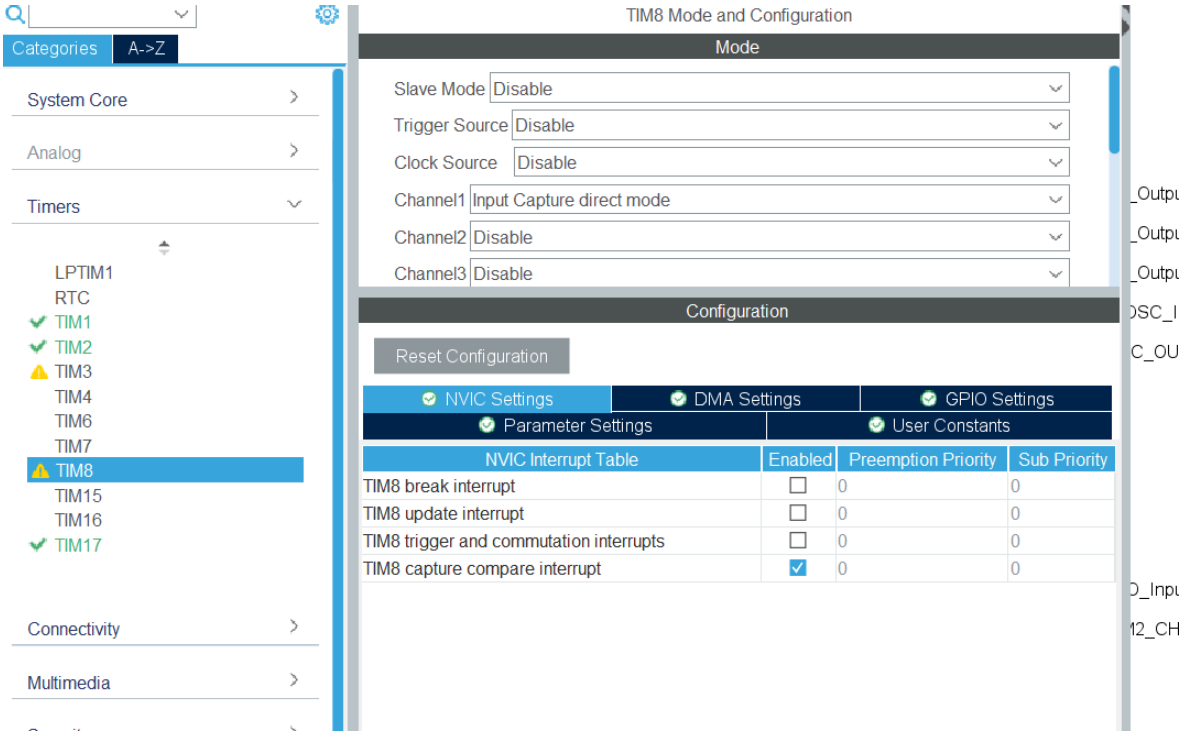
```
uint32_t fre,capture_val;
void HAL_TIM_IC_CaptureCallback(TIM_HandleTypeDef *htim)
{
    if(htim->Instance==TIM17)
    {
        capture_val=TIM17->CCR1;
        //capture_val=HAL_TIM_ReadCapturedValue(htim,TIM_CHANNEL_1);
        TIM17->CNT=0;
        fre=80000000/(80*capture_val);
    }
}
```

输入捕获555定时器产生的PWM波形



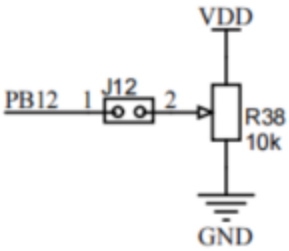
555定时器通过R40变阻器产生不同频率的PWM，可用内部的PA15引脚来输入捕获

原理和上面一样

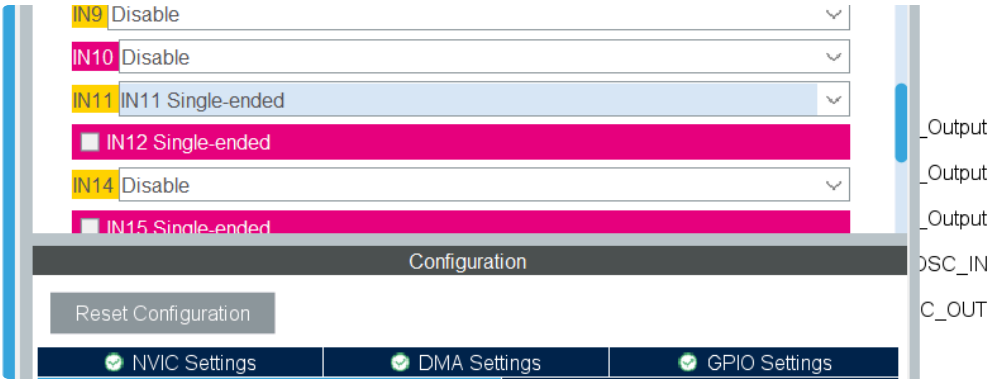



```
HAL_TIM_IC_Start_IT(&htim8,TIM_CHANNEL_1);
    if(htim->Instance==TIM8)
    {
        //capture_val=HAL_TIM_ReadCapturedValue(htim,TIM_CHANNEL_1);
        capture_val2=TIM8->CCR1;
        TIM8->CNT=0;
        Fre_NE555=80000000/(80*capture_val2);
    }
```

ADC测量：



配图电位器接的是 PB12，不是 PB2。STM32G431 上将 PB12 配置为 ADC1_IN11，选择 IN11 Single-ended 模式（单端采样以模拟地为参考；差分采样测量指定正、负输入端的电压差）。



在cubemx配置完后之后会生成一个adc文件，编译一下，将这个adc.h文件放在我们headfile.h里面

在 LCD 里面启动、获取、刷新：

注意：下面的入门示例省略了等待步骤。轮询采样应在 `HAL_ADC_Start` 后等待 `HAL_ADC_PollForConversion` 成功，再 `HAL_ADC_GetValue`；刚启动就直接读取，不能保证得到本次转换结果。

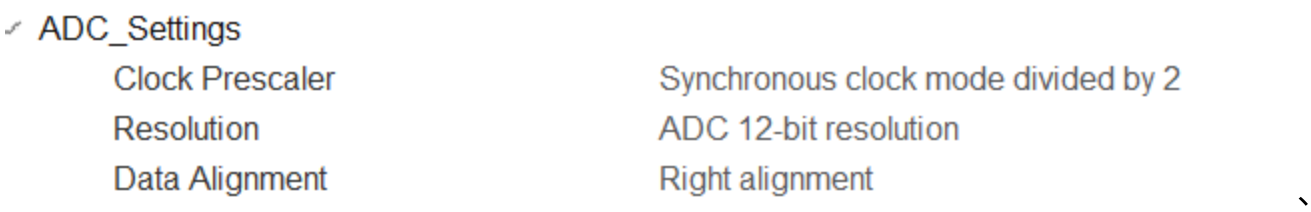
```
    HAL_ADC_Start(&hadc1);
    adc_val= HAL_ADC_GetValue(&hadc1);

        sprintf(text,"    adc_val:%5d    ",adc_val);
        LCD_DisplayStringLine(Line8,(u8*)text);
```

获取电压：

```
double get_vol(ADC_HandleTypeDef *hadc)
{
    HAL_ADC_Start(hadc);
    adc_val= HAL_ADC_GetValue(hadc);
    return 3.3*adc_val/4096;
}
//然后在LCD中直接显示
        sprintf(text,"    adc_val:%5.2f    ",get_vol(&hadc1));
        LCD_DisplayStringLine(Line8,(u8*)text);
```

为什么是除以4096?



因为这里ADC设置为12位量化

https://blog.csdn.net/weixin_72749746/article/details/137611347

ADC 校准（在初始化完成、开始采样之前进行）：

校准用于减小 ADC 本身的误差，不能保证一定读到精确的 3.3V；电压换算还取决于实际 VREF+，3.3 只是参考电压的假设值。12 位 ADC 有 4096 个量化等级，码值为 0～4095。

```
HAL_ADCEx_Calibration_Start(&hadc1,ADC_SINGLE_ENDED);
```

STM32 HAL库核心概念：句柄与指针规则

1. 命名缩写解析

- **前缀 `h`**：代表 `Handle`（句柄）。
  - `htim` = Handle of Timer（定时器句柄）
  - `hadc` = Handle of ADC（ADC句柄）
  - `huart` = Handle of UART（串口句柄）

2. 什么是“句柄 (Handle)”?

- **代码本质**：句柄是一个**结构体变量**（例如 `TIM_HandleTypeDef htim2;`），内部包含了该外设的：
  1. 物理寄存器基地址指针（`.Instance`）
  2. 初始化配置参数（`.Init`）
  3. 当前工作状态（Ready / Busy / Error 等）
- **作用**：HAL库函数通过接收不同外设的句柄，实现对不同硬件的统一化操作管理。

3. 句柄 (`htim2`) vs 底层寄存器 (`TIM2`)

名称格式	例子	属性	作用与用法
小写带 `h`	`htim2`	**软件结构体变量**	用于接收该外设句柄指针的 HAL 函数。 例: `HAL_TIM_PWM_Start(&htim2, ...);`

名称格式	例子	属性	作用与用法
全大写无 `h`	`TIM2`	**物理寄存器基地址**	用于直接操作底层寄存器，执行速度极快。 例: `TIM2->CCR2 = 50;`
关联关系	`htim2.Instance == TIM2`		

4. `&` (取地址符) 的使用铁律

HAL 库的操作函数形参通常要求传入**结构体的指针**。是否加 `&` 取决于当前变量的类型：

情况 A：必须加 `&` (取地址)

- **场景**：变量是全局定义的**结构体实体**（如在 main.c 中定义的 `TIM_HandleTypeDef htim2;`）。
- **原因**：函数需要指针，而你手里是实体，必须用 `&` 取出它的内存地址。
- **示例**：

```
HAL_TIM_PWM_Start(&htim2, TIM_CHANNEL_2);
HAL_ADC_Start(&hadc1);
```

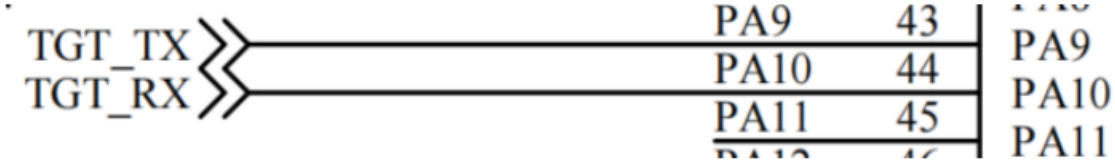
情况 B：坚决不能加 `&`

- **场景**：变量本身**已经是一个指针**（常见于回调函数或自定义函数的形参）。
- **原因**：它已经是地址了，再取地址就变成了“指针的指针”，会与函数形参类型不匹配，编译器通常给出诊断；若忽略诊断或强制转换，可能错误访问内存甚至触发 HardFault，并非必然直接崩溃。
- **示例**：

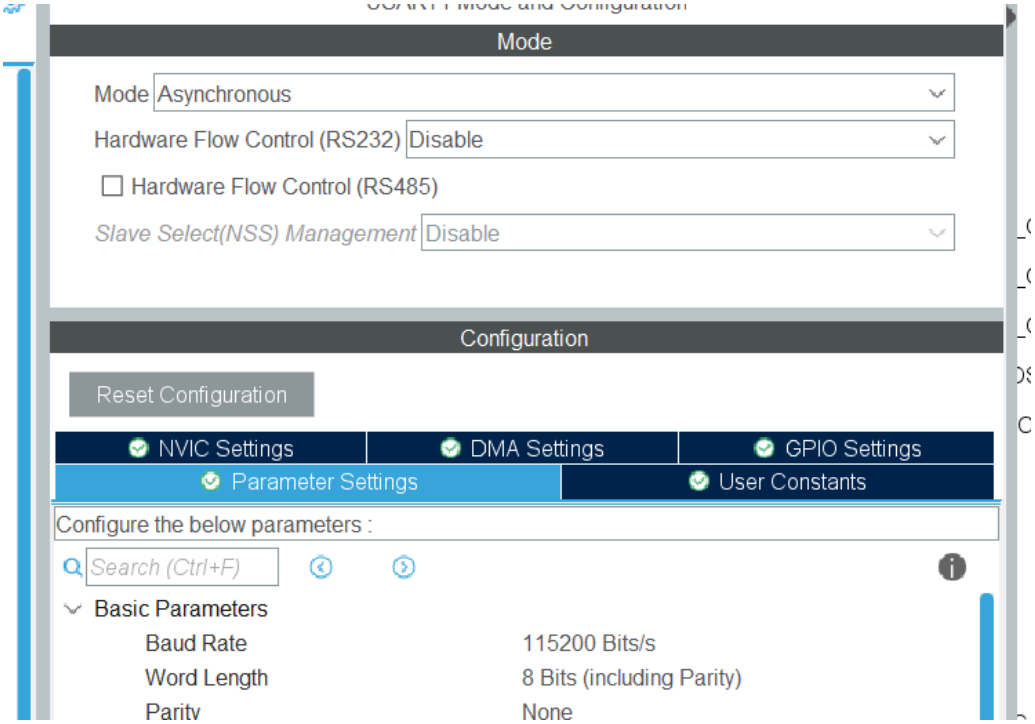
```
// 形参 htim 带有 *，说明它已经是指针
void HAL_TIM_IC_CaptureCallback(TIM_HandleTypeDef *htim)
{
    // 1. 调用HAL函数直接传指针变量名
    HAL_TIM_ReadCapturedValue(htim, TIM_CHANNEL_1);

    // 2. 访问内部成员必须用 -> 符号
    if(htim->Instance == TIM8)
    { ... }
}
```

串口：（推荐使用DMA）



STM32G431 使用 USART1 时，此处应将 PA9 配置为 USART1_TX、PA10 配置为 USART1_RX，不是 PA10/PA11；Mode 选择 Asynchronous 异步通信，记得使能中断。



发送字符串：

发送有效的、以 `\0` 结束的字符串时，HAL_UART_Transmit 的第三个参数可用函数 strlen，计算实际字符数、不含结束符；sizeof 得到对象的字节数（数组容量或指针大小）。二进制数据应传明确长度，不能一律使用 strlen。

```
    char text01[20];
    sprintf(text01,"hello world\r\n");
    HAL_UART_Transmit(&huart1,(uint8_t *)text01,strlen(text01),50);
    HAL_Delay(1000);//会阻塞
```

接收一个字符：

下面是在中断中回显的入门示例；阻塞发送会延长中断处理时间。如果计时用的 SysTick 无法抢占，注释中的 50ms 超时也不能保证按时退出。

```
//////////初始化中：
  HAL_UART_Receive_IT(&huart1,&rec_date,1);//相当于串口使能
  //////////中断中
  uint8_t  rec_date;
```

```
  void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
    if(huart->Instance==USART1)
    {
        HAL_UART_Transmit(huart,&rec_date,1,50);//最后的50是超时机制，最多等50ms就会跳出去
        HAL_UART_Receive_IT(huart,&rec_date,1);
    }

}
```

sprintf到底是什么

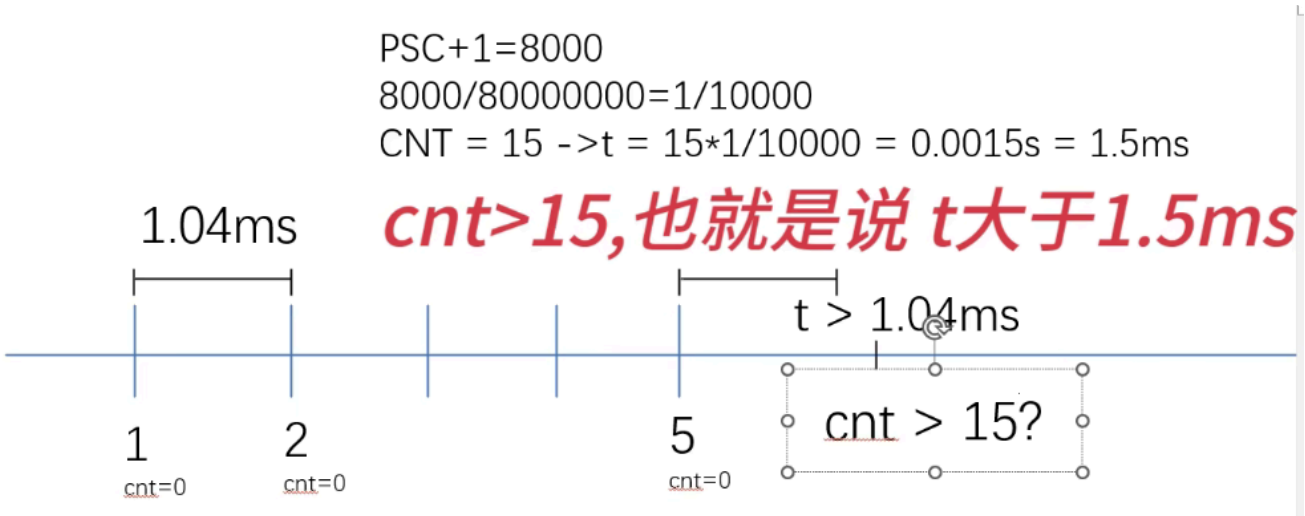
全称是 **string print formatted** 核心作用就是将这些数值**格式化**成我们可以阅读的**字符串**

回调函数：为什么要反复调用 `HAL_UART_Receive_IT`？

你在 `HAL_UART_RxCpltCallback` 中重复调用：`HAL_UART_Receive_IT(huart, &rec_date, 1);`

- **单次触发属性**：HAL 库的中断接收是“一次性”的。当它收完你指定的长度（比如 1 个字节）并触发回调函数后，HAL 会结束本次接收并关闭相关接收中断，不是关闭所有串口中断。
- **接力赛机制**：为了能持续接收后续数据，你必须在处理完当前字节后，立刻“重新布防”，告诉硬件：“请继续盯着，收完下一个再叫我”。

使用定时器接收字符串：



思路：以 8N1 为例，每个字符共 10bit，9600 波特率时约花费 1.04ms，不是 1.04s。超时判帧需要双方约定：每收到一个字节就清零计数，超过约定的空闲阈值后才认为一帧结束，不能仅因间隔超过 1.04ms 就认定字符串结束。下面 CNT>=20 的实际时间取决于 TIM4 的计数频率，并且需要先启动定时器；配图串口为 115200，不能与 9600 的计算混用。

```c
uint8_t rec_data,count0;
uint8_t rec_flag;
uint8_t rec_buff[20];
char send_buff[20];

void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
    if(huart->Instance==USART1)
    {
        TIM4->CNT=0;
        //HAL_UART_Transmit(huart,&rec_data,1,50);
        rec_flag=1;
        rec_buff[count0++]=rec_data;
        HAL_UART_Receive_IT(huart,&rec_data,1);
    }
}

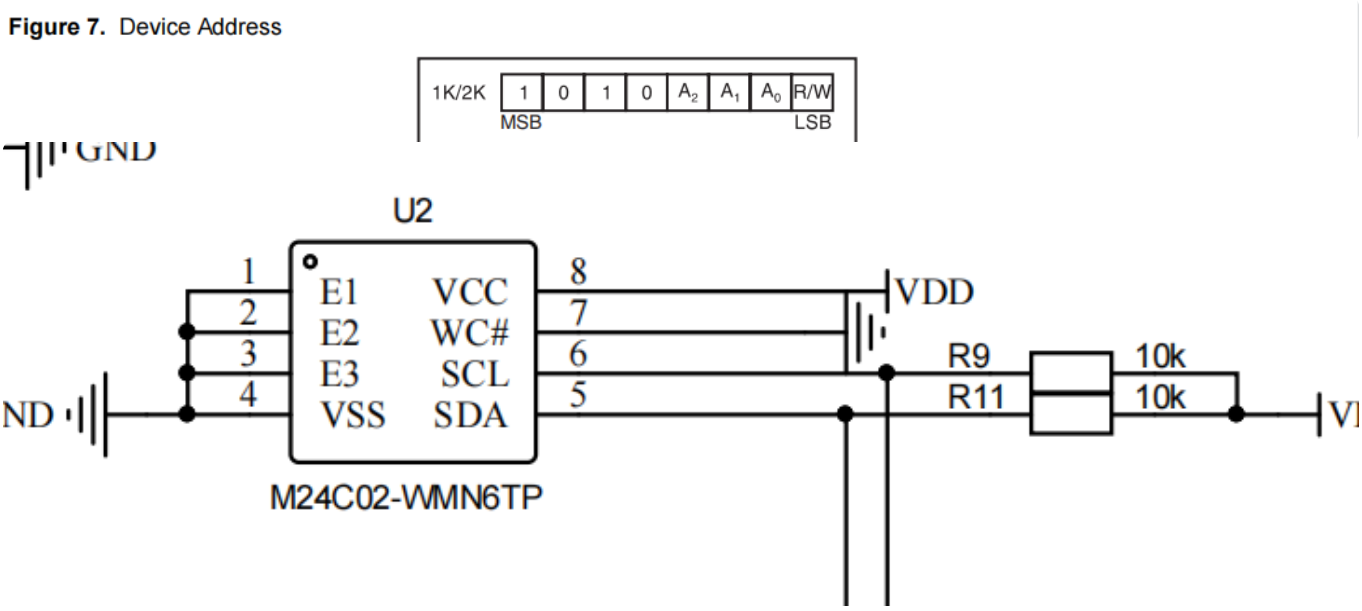
void uart_date_rec(void)
{
    if(rec_flag)
    {
        if(TIM4->CNT>=20)
        {
            if(rec_buff[0]=='a'&&rec_buff[1]=='b')
            {
                sprintf(send_buff,"abc");
                HAL_UART_Transmit(&huart1,(uint8_t
*)send_buff,strlen(send_buff),50);
            }
            else
            {
                sprintf(send_buff,"error");
                HAL_UART_Transmit(&huart1,(uint8_t
*)send_buff,strlen(send_buff),50);
            }
              // --- 核心清零操作 ---
            // 重置状态
            rec_flag=0;
            count0=0;
            memset(rec_buff, 0, sizeof(rec_buff));

        }
    }
}
```

这里有一个注意点：LCD 刷新耗时可能让主循环来不及判断帧结束。下面的固定缓冲区示例没有长度保护，接收超过 20 字节会越界；中断接收与主循环清零也可能互相打断，不能把这个入门示例直接当作可靠的连续收包方案。
除了这种定时器超时法，实际工程中更常使用的是 **串口空闲中断 (IDLE) + DMA**
【蓝桥杯嵌入式】串口DMA模式及不定长数据收发_嵌入式蓝桥杯dma-CSDN博客
！！！使用DMA会更简单更好用！！！

---

eeprom—AT24C02



AT24C02 的 7 位地址格式为 1010 A2 A1 A0，再加最低位读写位（读 1、写 0）。A2/A1/A0 全接地时，7 位地址是 0x50（1010000），总线上发送的写/读地址字节是 0xA0/0xA1；原来的 0101000 位序有误。

```
//1是读取 0是写
void eeprom_write(uint8_t addr,uint8_t dat)
{
    I2CStart();
    I2CSendByte(0xa0);
    I2CWaitAck();
    I2CSendByte(addr);
    I2CWaitAck();
    I2CSendByte(dat);
    I2CWaitAck();
    I2CStop();

    HAL_Delay(20);
}
```

```
uint8_t eeprom_read(uint8_t addr)
{
//第一步将AT24C02内部指针对准要读取的位置
    I2CStart();
    I2CSendByte(0xa0);
    I2CWaitAck();
    I2CSendByte(addr);
    I2CWaitAck();
    I2CStop();

//正式开始读数据
    I2CStart();
    I2CSendByte(0xa1);
    I2CWaitAck();
    uint8_t dat=I2CReceiveByte();
    I2CSendNotAck();
    I2CStop();

    return dat;
}
```

在初始化中：

```
I2CInit();
eeprom_write(0,10);
uint8_t dat=eeprom_read(0);
```

RTC实时时钟

比较简单这里就水了，当达到闹钟时 led8 就会亮（需先配置闹钟 A 并使能闹钟中断，仅定义回调不会自动触发）。
日期字段注意：下方原示例的 `sDate.WeekDay` 是星期，不是月内日期；显示年月日应使用 `sDate.Date`。`sDate.Year` 为两位年份，显示完整年份时还需按使用的世纪换算。

```
char text[20];
  RTC_TimeTypeDef sTime = {0};
  RTC_DateTypeDef sDate = {0};
  uint8_t led8state;
void show_lcd(void)
{
```

```
      HAL_RTC_GetTime(&hrtc,&sTime,RTC_FORMAT_BIN);
      HAL_RTC_GetDate(&hrtc,&sDate,RTC_FORMAT_BIN);
    sprintf(text,"  Hello,World!");
    LCD_DisplayStringLine(Line0,(uint8_t *)text);
    sprintf(text,"  %2d:%2d:%2d    ",sTime.Hours,sTime.Minutes,sTime.Seconds);
    LCD_DisplayStringLine(Line1,(uint8_t *)text);
    sprintf(text,"  %d-%d-%d      ",sDate.Year,sDate.Month,sDate.WeekDay);
    LCD_DisplayStringLine(Line2,(uint8_t *)text);

    show_led(8,led8state);
}
void HAL_RTC_AlarmAEventCallback(RTC_HandleTypeDef *hrtc)
{
    led8state=1;
}
```

完结撒花！！！ ❀❀❀

希望学弟学妹们能用尽量少的时间高效备赛，在兼顾其他比赛的同时考出好成绩，也替学长弥补一下当年没能进国赛的小遗憾 (╥﹏╥)

考试不难、但也补药大意，对于任何比赛、生活来讲：***心浮气躁是大忌。***